

Ensembles: bagging, random forests, boosting

Many models vote or average. Bootstrap samples, random features and reweighting make them differ.

THE QUESTION

When can a group of mediocre models, combined, beat the best single one?

THE INTUITION

Models that beat chance and are wrong on different examples outvote each other's mistakes. Bagging and forests make trees differ by resampling rows and features; boosting trains each model on the mistakes of the ones before.

KEY TAKEAWAY

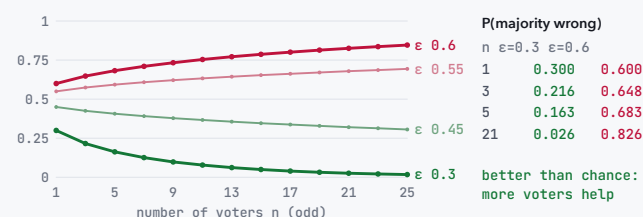
Average many deep trees to cut variance (bagging, forests); add weighted weak learners to cut bias (boosting). Members must beat chance and not all fail together.

one dataset → bootstrap the rows → sample features per split → or reweight the mistakes → vote or average

1 Vote for classes, average for numbers

An ensemble combines models $h_1, \dots, h_n$. The slides' examples: two of three spam filters say spam, so the vote says spam; three price models (\$420K, \$450K, \$430K) average to \$433.3K. It beats its members only if they are **accurate** (error below 0.5) and **diverse** (wrong on different examples). Limited data, greedy training and a model family too simple for the true boundary all leave room for it.

2 Independent voters: the majority errs less

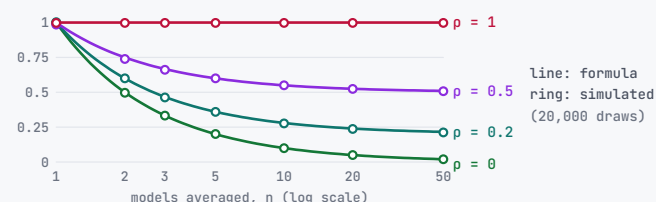


With n independent classifiers, each wrong with probability ϵ , the vote is wrong when more than half are:

$$P(\text{vote wrong}) = \sum_{k > n/2} \binom{n}{k} \epsilon^k (1 - \epsilon)^{n-k}$$

Three at $\epsilon = 0.3$: $3 \cdot 0.3^2 \cdot 0.7 + 0.3^3 = 0.216$. At $\epsilon = 0.6$, more voters make it worse.

3 Correlated models cancel less



Members trained on one dataset make related errors. For n models with error variance σ^2 and pairwise correlation ρ :

$$\text{Var}\left(\frac{1}{n} \sum_i h_i\right) = \rho \sigma^2 + (1 - \rho) \frac{\sigma^2}{n}$$

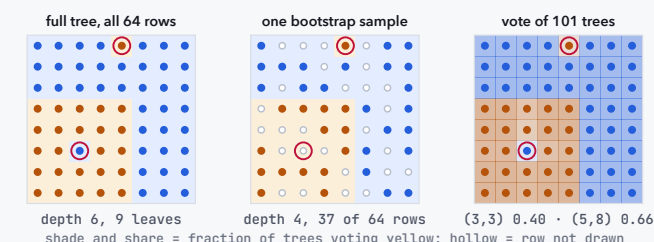
More models shrink only the second term; identical models ($\rho = 1$) gain nothing. (A standard result; the slides say it in words.)

4 Bootstrap samples hold about 63% of rows



Draw n rows with replacement. A row is missed with probability $(1 - \frac{1}{n})^n \rightarrow \frac{1}{e} \approx 0.368$, so about 63.2% of rows appear. For 150 iris rows the notebook gives $0.632 \cdot 150 = 94.8$ (exactly 95.0). The out-of-bag (OOB) rows test that model for free.

5 Bagging: a full tree per sample, then a vote



The slides' grid: yellow is $x_1 \leq 5, x_2 \leq 5$, with (3,3) and (5,8) flipped (ringed). A full tree boxes in each outlier; every sample grows a different tree. Only trees whose sample holds an outlier vote for it (61 and 67 of 101), so the vote keeps both, by about 60 to 40. Bagging cuts variance; wrong labels stay. The slides' 5-tree vote dropped both boxes (chance 0.26 per outlier).

6 Bagging with an out-of-bag score

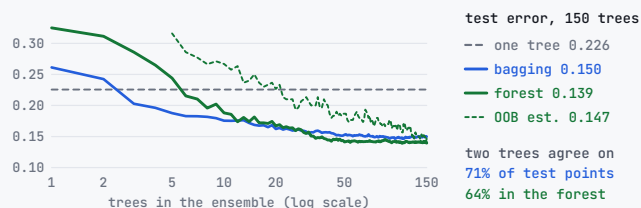
```
def bag(X, y, B, fit, rng):
    n = len(y); models = []
    yes, cnt = np.zeros(n), np.zeros(n)
    for b in range(B):
        idx = rng.integers(0, n, n)
        h = fit(X[idx], y[idx])
        out = np.setdiff1d(range(n), idx)
        yes[out] += h(X[out])
        cnt[out] += 1
    models.append(h)
    ok = cnt > 0
    share = yes[ok] / cnt[ok]
    oob = np.mean((share > 0.5) != y[ok])
    return models, oob
```

fit returns a predict function. With 150 forest trees on the panel 7 data: test error 0.137, OOB 0.153.

Decorrelate the trees, or fix the mistakes

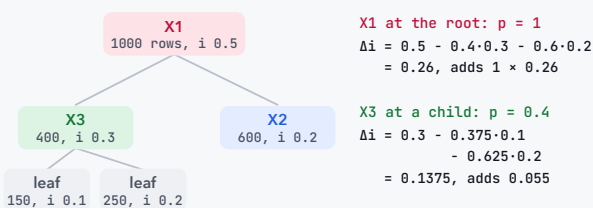
Forests sample features at every split; boosting builds models in sequence and reweights their mistakes.

7 Random forests also sample features at each split



For $b = 1, \dots, B$: draw a bootstrap sample and grow a tree whose every split considers only m of the M features, drawn at random ($m = \sqrt{M}$ or $\log_2 M$; iris: $M = 4, m = 2$). Our data (illustrative): 300 rows, 8 features, labels from a noisy linear rule, $m = 2$. A forest tree alone is worse (test error 0.296 against 0.257 for a bagged tree), but the trees agree less, so the forest ends lower. OOB error tracks test error with no held-out data.

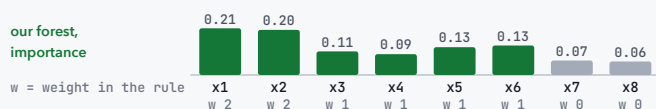
8 Importance adds up impurity drops



Each split on X_j adds $p(t)\Delta i(t)$, with $p(t) = N_t/N$ and $\Delta i(t) = i(t) - \frac{N_{tL}}{N_t}i(t_L) - \frac{N_{tR}}{N_t}i(t_R)$; average over the B trees:

$$\text{Imp}(X_j) = \frac{1}{B} \sum_{b=1}^B \sum_{t \in T_b} \mathbb{1}(j_t = j) p(t) \Delta i(t)$$

scikit-learn scales each tree's scores to sum to 1 first. The notebook's iris forest: petal width 0.544, petal length 0.323, sepal length 0.099, sepal width 0.034. On our data (below) the weight-2 features lead, yet pure noise still scores about 0.06: deep trees split on noise near the leaves.



11 Pause and think

Bag depth-1 stumps? Little gain: stumps have high bias and low variance. Boost them instead.
Need a validation set for a forest? OOB error does that job: each row is scored by trees that never saw it.

Run boosting rounds in parallel? No: round m needs the weights left by round $m - 1$.
Five copies of one tree? $\rho = 1$: the vote is that tree, however many copies.

A learner with err 0.7? $\alpha = -0.42$: AdaBoost uses it flipped. At err 0.5, $\alpha = 0$.
Does bagging reduce bias? Hardly: the average has about the bias of one tree; the gain is in variance.

vote members that beat chance · bootstrap rows and sample features to decorrelate · reweight mistakes to boost weak learners

Before: the decision trees II, bias and variance, and cross-validation sheets. Source: lecture slides "Ensemble learning" and the boosting-explanation, ensemble-feature-importance, ensemble-representation and binomial notebooks (nipunbatra.github.io/ml-teaching). The grid and the ten AdaBoost points are recreated from the slide figures; the 8-feature data and all runs are ours, seeded; notebook numbers are quoted.

9 AdaBoost trusts each learner by its error

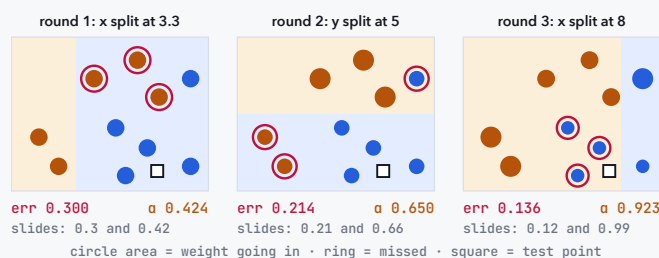


Bagging trains strong learners in parallel to cut variance; boosting trains weak ones (just above chance) in sequence to cut bias. AdaBoost, labels $\pm 1, w_i = 1/N$. For $m = 1, \dots, M$: fit h_m to the weighted data, then

$$\text{err}_m = \frac{\sum_{\text{wrong}} w_i}{\sum_i w_i}, \quad \alpha_m = \frac{1}{2} \ln \frac{1 - \text{err}_m}{\text{err}_m}$$

Multiply wrong w_i by e^{α_m} , right ones by $e^{-\alpha_m}$, normalise. Predict $\text{sign}(\sum_m \alpha_m h_m(x))$. On the panel 7 data, 100 boosted stumps score 0.134 (one stump: 0.306), level with the forest.

10 The slides' ten points, three stumps



Round 1 misses 3 of 10: err 0.3, $\alpha_1 = \frac{1}{2} \ln(0.7/0.3) = 0.424$. The misses rise to 1/6, the rest fall to 1/14.
 Round 2 misses three of weight 1/14: err 3/14, $\alpha_2 = 0.650$. Round 3: err 3/22, $\alpha_3 = 0.923$. Test point: $-0.424 - 0.650 + 0.923 = -0.150$, blue. The slides round the weights down (0.10, 0.16, 0.04 sum to 0.94), hence err 0.12 and α 0.99; still blue. The stumps are drawn by hand: in round 1 a split near $x = 4.75$ would miss only 2 points.